

What is JavaScript?



프로그래밍 언어, JavaScript, 그리고 개발 환경

목차

01	프로그래밍
02	프로그래밍 언어와 번역기
03	JIT 컴파일
04	JavaScript
05	V8 엔진
06	런타임과 도구
07	mise

01

Section

프로그래밍이란 무엇일까요?

사람이 컴퓨터에게 일을 시키는 방법

프로그래밍

사람과 컴퓨터의 커뮤니케이션

프로그래밍이란 컴퓨터에게 특정한 작업의 실행을 요구하는 일종의 커뮤니케이션

본질

그래서 본질은
문제 해결입니다.

사람이 생각한 해결 방법을
컴퓨터가 이해하고 실행할 수 있는 형태로 표현하는 과정

문제 해결에 필요한 다섯 가지 사고

- 1 문제를 명확하게 이해
- 2 복잡한 문제를 작은 문제로 분해
- 3 필요한 자료를 정리하고 구분
- 4 반복되는 패턴을 찾아 추상화
- 5 해결 절차를 작은 단위의 명령으로 구체화

컴퓨터의 관점

사람에게 당연한 것이 컴퓨터에게는 아닙니다.

사람에게는

"차선을 따라 일정한 속도로 주행하라"

컴퓨터에게는

- 차선이 무엇인지
- 차선을 어떻게 인식하는지
- 일정한 속도는 어느 정도인지
- 이탈을 어떻게 판단하는지

그런데

**컴퓨터는 결국
0과 1로 이루어진
기계어만 직접 이해합니다.**

→ 그렇다면 사람이 정리한 절차를 어떻게 전달할까요?

02

Section

프로그래밍 언어와 번역기

컴파일러, 인터프리터, 그리고 그 사이

프로그래밍 언어

사람과 컴퓨터를 이어주는 인공지능

사람이 읽고 작성할 수 있는 문법과 규칙으로 코드를 작성하고,
이를 컴퓨터가 이해할 수 있는 형태로 변환합니다.

코드 작성

번역

실행

번역기

소스 코드를 실행 가능한 형태로

컴파일러

Compiler

코드 전체를 분석한 뒤, 실행 가능한 형태로 변환

인터프리터

Interpreter

실행하는 과정에서 번역과 실행을 함께 수행

컴파일 언어

미리 컴파일해서 실행 파일을 만듭니다.

장점

- 실행 속도가 빠름
- 실행 전에 일부 오류 발견

단점

- 수정할 때마다 다시 컴파일
- OS마다 따로 빌드 필요

인터프리터 언어

런타임에 번역과 실행이 함께 일어납니다.

장점

- 작성 후 바로 실행 가능
- OS에 구애받지 않음

단점

- 실행 속도가 비교적 느림
- 오류가 런타임에 발견됨

현실

하지만 둘의 경계는 점점 흐려지고 있습니다.

현대의 언어와 실행 환경은
컴파일과 인터프리트 방식을 혼합해서 사용하는 경우가 많습니다.

→ 그 절충안이 바로 JIT 컴파일

03

Section

JIT 컴파일

두 세계의 절충안

왜 등장했을까?

각자의 한계를 넘기 위해서.

컴파일러

빠르지만 OS마다 다시 빌드해야 함

인터프리터

어디서든 동작하지만 속도가 느림

JIT

환경에 자유로우면서 빠르게 실행

JIT

자주 실행되는 코드만 미리 기계어로 바꿉니다.

Just-In-Time Compilation

코드 실행 도중에 필요한 부분을 그때그때 기계어로 변환하는 방식.
전체를 미리 컴파일하지도, 매번 한 줄씩 번역만 하지도 않는 절충안입니다.

동작 방식

JIT은 이렇게 작동합니다.

- 1 인터프리터로 코드 실행 시작
- 2 자주 호출되는 코드를 관찰
- 3 핫 코드를 기계어로 컴파일
- 4 다음 실행부터 기계어 재사용
- 5 가정이 깨지면 다시 인터프리터로

장 단점

빠르지만, 단점도 존재합니다.

장점

- 환경에 종속되지 않음
- 컴파일 언어에 가까운 속도
- 런타임 정보로 최적화

단점

- 초반 워밍업 구간이 느림
- 메모리 사용량 증가
- 구현이 매우 복잡함

대표 사례

JIT은 이미 어디서나 동작합니다.

V8

JavaScript

HotSpot

Java

PyPy

Python

04

Section

JavaScript

브라우저에서 시작된 언어

1995년, Netscape

가벼운 웹 인터랙션을
위해 태어났습니다.

1995

폼 검증, 마우스 호버 같은
정말 가벼운 인터랙션을 위한 언어

표준

ECMAScript와 JavaScript는 다릅니다.

ECMAScript

언어 자체의 문법과 동작을 정의한 **표준 명세**

JavaScript

ECMAScript + 환경 API
(DOM, 네트워크 요청, 저장소 등)

실제로는 이렇게 생겼습니다.

15.2 Function Definitions

Syntax

```
FunctionDeclaration[Yield, Await, Default] :  
    function BindingIdentifier[?Yield, ?Await] ( FormalParameters[~Yield, ~Await] ) { FunctionBody[~Yield, ~Await] }  
    [+Default] function ( FormalParameters[~Yield, ~Await] ) { FunctionBody[~Yield, ~Await] }  
  
FunctionExpression :  
    function BindingIdentifier[~Yield, ~Await] opt ( FormalParameters[~Yield, ~Await] ) { FunctionBody[~Yield, ~Await] }  
  
FunctionBody[Yield, Await] :  
    FunctionStatementList[?Yield, ?Await]  
  
FunctionStatementList[Yield, Await] :  
    StatementList[?Yield, ?Await, +Return] opt
```


ES 6 (2015)

JavaScript의 분기점이 된 버전.

단일 버전 기준 가장 큰 폭의 업데이트.
기존 JavaScript의 고질적인 문제들을 한꺼번에 해결했습니다.

ES6 이전과 이후의 코드는 사실상 다른 언어처럼 보일 정도

ES6 주요 문법

이런 것들도 도입되었습니다.

let / const

var의 스코프 문제 해결

화살표 함수

함수 작성 단순화, this 바인딩 완화

class

프로토타입을 더 직관적으로 표현

import / export

표준 모듈 시스템 도입

Promise

콜백 중첩 문제 정리

05

Section

V8 엔진

Chrome과 Node.js의 심장

V8

Google이 만든 오픈소스 JavaScript 엔진.



공개 2008년, Chrome과 함께

언어 C++

역할 JavaScript 코드를 받아 빠르게 실행

V8 실행 과정

코드는 네 단계를 거쳐 실행됩니다.

01

파싱

Parser → AST

02

바이트코드

Ignition

03

최적화

TurboFan

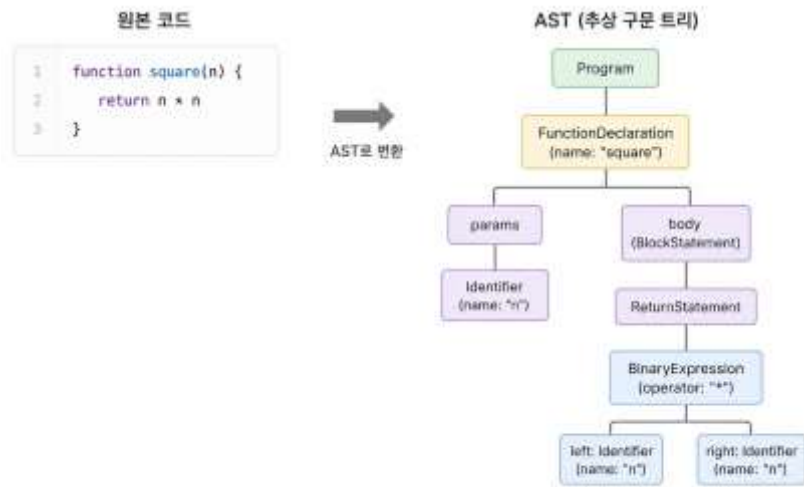
04

캐싱

기계어 재사용

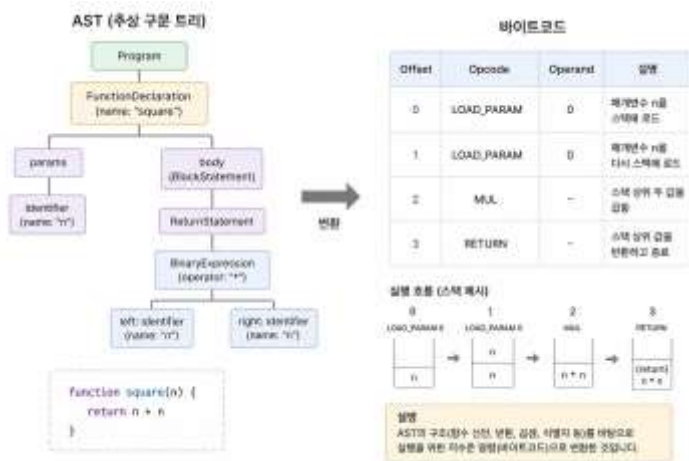
Step 01 · 파싱

코드를 구조화된 트리로 정리합니다.



Step 02 · 바이트코드

Ignition이 더 단순한 명령어로 변환합니다.



Step 03 · 최적화

핫 코드는 TurboFan이 기계어로 컴파일합니다.



Step 04 · 캐싱

한 번 만든 기계어는 다시 사용합니다.

이후 동일한 코드가 다시 실행될 때는,
컴파일된 기계어를 그대로 사용합니다. 번역 과정이 생략되어 매우 빠릅니다.

→ V8 내부에서 JIT이 실제로 구현되는 흐름

그런데 V8 혼자서는

파일도, 네트워크도
다룰 수 없습니다.

→ 브라우저 밖에서 JavaScript를 실행하려면?

06

Section

런타임과 도구

Node.js, bun, npm, pnpm

2009년, Ryan Dahl

V8을 브라우저 밖으로 꺼낸 실행 환경.

어디서든 JavaScript를 실행할 수 있도록 만든 환경이 Node.js입니다.
JavaScript로 웹 서버, CLI 도구, 빌드 도구까지 만들 수 있게 되었습니다.

Node.js 구성

V8에 libuv와 Noew.js API를 결합해 만든 런타임

비동기 I/O를 처리

libuv

fs, http, path 등

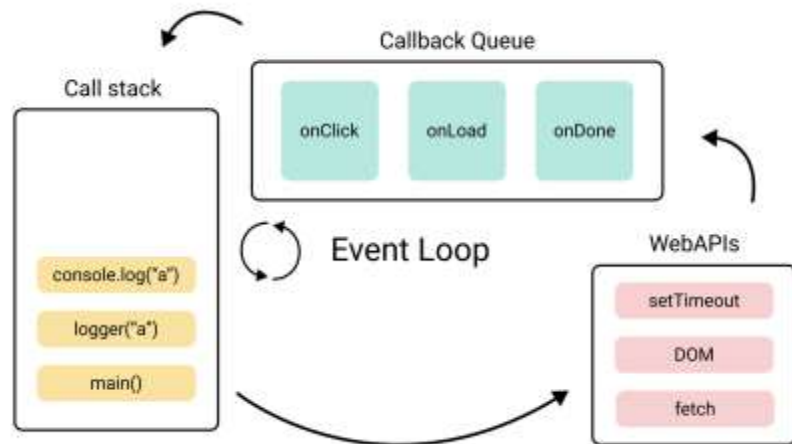
표준 라이브러리

V8 엔진

이벤트 루프

싱글 스레드인데 어떻게 동시에?

- 1 오래 걸리는 작업을 만나도 멈추지 않음
- 2 작업을 백그라운드로 넘기고 다음 코드 실행
- 3 완료되면 이벤트 루프가 결과를 전달



패키지 매니저

npm과 pnpm은 저장 방식이 다릅니다.

npm

프로젝트마다 `node_modules`를 만들고 그 안에 패키지 파일을 복사합니다.

→ 디스크 용량 낭비

pnpm

전역 저장소에 한 번만 저장하고, 각 프로젝트는 링크만 사용합니다.

→ 용량 절약 + 설치 속도 향상

bun (2022)

Node.js의 새로운 대안.

언어	Zig
엔진	JavaScriptCore (Safari)
구성	런타임 + 번들러 + 테스트 + 패키지 매니저

왜 J S C 인가?

용도에 따라 엔진 강점이 다릅니다.

JavaScriptCore

- 시작 속도와 메모리 효율에 최적화.
- CLI, 빌드, 테스트처럼 짧게 시작해 빠르게 끝나는 작업에 강함.

V8

- 장기 실행 환경의 JIT 최적화에 강점
- 서버나 무거운 연산에서는 V8 기반 Node.js가 더 빠를 수 있음.

정리

V8과 Node.js는 담당이 다릅니다.

V8

JavaScript 코드를 해석하고 실행하는 엔진

Node.js

V8 + libuv + 표준 라이브러리 시스템 작업까지 가능한 실행 환경

07

Section

mise

여러 도구의 버전을 한 번에

기존의 불편함

도구마다 버전 매니저가 따로 있었습니다.

Node.js

→

nvm

Python

→

pyenv

Ruby

→

rbenv

Java

→

jenv

mise

여러 도구의 버전을 하나의 명령어로.

개발 도구의 버전을 통합적으로 관리할 수 있게 해주는 버전 매니저입니다.

```
$ mise install node@20  
$ mise install python@3.12  
$ mise install java@21
```

프로젝트 단위 관리

디렉토리에 들어가면 버전이 자동 전환됩니다.

.mise.toml 파일에 사용할 버전을 명시하면,
해당 프로젝트 디렉토리에서 자동으로 그 버전이 활성화됩니다.

→ 더 이상 여러 매니저를 따로 설치할 필요 없음

감사합니다.